

prevIA - Documento tecnico base

Arquitetura, produto, rotas, dados, jobs, billing, campanhas, parceiros e Bolao da Copa

Documento gerado a partir do ZIP portable.zip

2026-07-04

Indice

1 Controle do documento

Este documento foi gerado a partir da analise estatica do pacote portable.zip, extraido no ambiente de trabalho em 2026-07-04. O objetivo e servir como base tecnica para pessoas e IAs que precisem entender, auditar, evoluir ou operar o prevIA.

O documento evita reproduzir valores de variaveis de ambiente, chaves, cookies, tokens e segredos. Quando arquivos sensiveis aparecem no ZIP, eles sao citados apenas como risco operacional.

Escopo coberto: backend FastAPI, frontend React/Vite, rotas publicas, app logado, Admin/Ops, billing Stripe, campanhas, parceiros, product snapshots, odds/modelos, image import, telemetria e Bolao da Copa 2026.

2 Resumo executivo

O prevIA e uma plataforma de inteligencia esportiva para futebol. O produto combina dados historicos, odds, modelos probabilisticos e uma camada narrativa para ajudar o usuario a avaliar jogos, mercados e oportunidades com uma linguagem mais simples e responsavel.

A arquitetura atual possui quatro grandes superficies: uma landing publica multilíngue, um app de produto com autenticação e créditos, um painel administrativo/ops para pipeline e auditoria, e um modulo publico de Bolao da Copa que atua como produto viral/topo de funil.

O backend e um monolito modular em FastAPI com PostgreSQL como banco principal. O frontend e uma SPA React/Vite/TypeScript servida por Firebase Hosting, com chamadas para a API em Cloud Run. A infraestrutura descrita no ZIP aponta para Google Cloud Run em southamerica-east1, Cloud SQL, Firebase Hosting, Cloud Scheduler e Secret Manager.

A parte de monetizacao usa planos, créditos, assinatura Stripe e campanhas de trial/beta. O codigo ja possui uma estrutura de elegibilidade de desconto pos-trial, mas o checkout Stripe, no

estado analisado, ainda permite apenas codigos promocionais manuais e nao aplica automaticamente o desconto da campanha.

3 Mapa mental do sistema

Usuario publico

- > Firebase Hosting / React SPA
- > paginas publicas: landing, glossario, parceiros, campanhas, Bolao da Copa
- > API publica FastAPI: /public/*, /access/campaigns/*, /public/worldcup-pool/*

Usuario logado

- > /app
- > autenticacao /auth/*
- > indice de jogos /product/index
- > consumo de creditos /access/usage e /access/reveal
- > assinatura /billing/*
- > analise manual e importacao de imagem /product/manual-analysis/*

Admin/Ops

- > /admin
- > autenticacao e capacidade admin via /auth/me
- > odds, ligas, auditoria, usuarios, campanhas, parceiros
- > jobs e pipeline via /admin/ops/*

Cloud Scheduler / operacoes internas

- > POST /internal/ops/jobs/run com X-Ops-Trigger-Token
- > dispatcher de jobs
 - > odds_refresh
 - > odds_resolve_batch
 - > models_ensure_1x2_v1
 - > snapshots_materialize
 - > pipeline_run_all
 - > jobs do Bolao da Copa

Providers externos

- > API-Football: fixtures, resultados, ligas, times
- > The Odds API: odds 1x2 e mercados relacionados
- > OddsPapi: enriquecimento controlado de odds
- > Stripe: checkout, assinaturas, webhooks
- > OpenAI Vision: leitura de prints no Image Import

4 O que o prevIA faz

4.1 Produto principal

- Lista jogos de futebol com informacoes de competicao, times, horario, odds e contexto.
- Controla acesso por plano e por creditos diarios.
- Permite revelar analises de jogos usando creditos.
- Calcula probabilidades 1x2 por modelo estatistico e compara com odds disponiveis.
- Classifica oportunidades quando a odd parece compensar o risco dentro das regras de margem, confianca e cobertura.
- Gera narrativa contextual em linguagem natural para explicar a leitura do jogo.
- Permite analise manual de jogos e importacao de prints de casas de apostas para usuarios elegiveis.

4.2 Modulos complementares

- Landing publica, glossario e conteudos educativos.
- Campanhas beta/trial com slug publico e concessao temporaria de plano.
- Oferta pos-trial com elegibilidade de desconto a ser conectada automaticamente ao Stripe.
- Modulo de parceiros/afiliados com candidaturas, conversao, links de campanha e atribuicoes.
- Bolao da Copa 2026 com criacao publica, convites, palpites, ranking, organizador e jobs de resultados.
- Admin/Ops para operar ligas, odds, resolucao de times, materializacao de snapshots, auditoria e jobs. # Stack tecnica

4.3 Backend

O backend roda em Python 3.11 com FastAPI e Uvicorn. Dependencias observadas no backend/requirements.txt:

```
fastapi>=0.110
uvicorn[standard]>=0.27
httpx>=0.24
python-dotenv>=1.0
psycpg[binary]>=3.1
numpy>=1.26
scikit-learn>=1.4
google-auth>=2.38
email-validator>=2.2.0
requests>=2.32.0
stripe>=12.0.0
python-multipart>=0.0.9
Pillow>=10.0
```

4.4 Frontend

O frontend usa React 18, React Router 6, TypeScript e Vite. Dependencias principais observadas em frontend/package.json:

```
@stripe/react-stripe-js: 4.0.2
@stripe/stripe-js: 7.9.0
react: 18.3.1
react-dom: 18.3.1
react-router-dom: 6.30.3
```

Dependencias de desenvolvimento principais:

```
@types/react: 18.3.5
@types/react-dom: 18.3.0
@vitejs/plugin-react: 4.7.0
typescript: 5.5.4
vite: 5.4.21
```

4.5 Banco e infraestrutura

- Banco principal: PostgreSQL, organizado por schemas como raw, core, odds, product, app, auth, billing, access, ops, telemetry, partnership e worldcup_pool.
- Deploy backend: Google Cloud Run, servico previa-api, regioao southamerica-east1.
- Banco gerenciado: Cloud SQL, com DATABASE_URL vindo de Secret Manager no deploy.
- Frontend: Firebase Hosting com SPA rewrite para index.html e rewrite de API para Cloud Run.
- Jobs: Cloud Scheduler chamando /internal/ops/jobs/run com token interno.
- Pagamentos: Stripe Checkout, webhooks sandbox/live e catalogo de planos/precos no banco. # Estrutura do repositorio

A estrutura analisada tem um repositorio portable com backend, frontend, arquivos de deploy e scripts. Resumo das areas principais:

backend/

app.py Entrada FastAPI e inclusao de routers

Dockerfile Imagem Cloud Run

requirements.txt Dependencias Python

cloudbuild*.yaml Builds/deloys no Google Cloud

docs/ Documentacao operacional interna

migrations/ Migrations SQL historicas

src/

access/ Campanhas, creditos, entitlements, revelacoes

api/ Admin principal, catalogo, usuarios, campanhas, parceiros, ops

auth/ Usuarios, sessoes, senhas, Google login, acesso interno

billing/ Planos, precos, Stripe checkout, webhooks, mudancas de plano

core/	Configuracoes, banco, modelos basicos
image_import/	Validacao, OCR/vision, resolucao de prints
integrations/	API-Football, The Odds API, OddsPapi, SMTP
models/	Score engine, hist5 decay, snapshots, oportunidades
odds/	Pipeline de odds, resolucao, catalogo, auditoria
ops/	Dispatcher e runners de jobs
partnership/	Candidaturas, contratos, links e atribuicoes
product/	Indice publico/logado, analise manual, snapshots
routes/	Rotas publicas, auth, billing, access, product, telemetry, Bolao
frontend/	
src/	
routes.tsx	Rotas publicas e gates /app e /admin
product/	Produto logado, conta, assinatura, analise manual
admin/	Admin SPA e paginas de operacao
public/	Landing, glossario, parceiros, campanhas, Bolao
api/	Clientes HTTP do frontend
firebase.json	Hosting e rewrites
package.json	Dependencias React/Vite
proxy/	
Cloud Run proxy local ou utilitario operacional	

Inventario textual aproximado, excluindo .git, node_modules, dist, .firebase, caches e bytecode:

```
.py    arquivos= 180 linhas= 68686
.json  arquivos= 170 linhas= 47001
.ts    arquivos= 88 linhas= 19659
.tsx   arquivos= 76 linhas= 35520
.sql   arquivos= 67 linhas= 4431
.css   arquivos= 10 linhas= 20058
.ps1   arquivos= 6 linhas= 660
.yaml  arquivos= 4 linhas= 354
.html  arquivos= 2 linhas= 29
.md    arquivos= 2 linhas= 852
```

5 Backend FastAPI

backend/app.py cria a aplicacao FastAPI, configura CORS, valida guardrails de producao e registra todos os routers. Em producao, o app bloqueia inicializacao em casos de configuracao perigosa, como bypass admin ativo, auto-login de produto ativo, admin auth desligado ou trigger manual de ops sem token.

Routers principais registrados:

- `/public/*`: leads beta, contato e candidaturas de parceiros.
- `/auth/*`: sessao, cadastro, login, Google login/link, preferencias e senha.
- `/access/*`: campanhas, uso diario, revelacao de jogos e reset de teste em ambiente interno.
- `/billing/*`: catalogo, checkout Stripe, assinatura, mudanca/cancelamento/retomada e webhooks.
- `/product/*`: indice de jogos, ligas, analise manual e image import.
- `/odds/*`: endpoints legados ou auxiliares de eventos/quote/matchups.
- `/admin/*`: dashboard, odds, catalogo, ops, usuarios, campanhas, parceiros, auditoria e telemetria.
- `/internal/ops/*`: execucao interna de jobs via Scheduler.
- `/partner/*`: painel do parceiro autenticado.
- `/telemetry/events`: coleta de eventos anonimos/publicos.
- `/public/worldcup-pool/*`: Bolao da Copa, se feature flag estiver habilitada. # Frontend React/Vite

O frontend e uma SPA com roteamento por React Router. Existem tres superficies: publica, produto logado e admin. O idioma aparece no path publico (`/:lang`) com suporte observado para PT, EN e ES em partes do produto.

Rotas publicas e gates principais:

```
/                -> redireciona para /pt
/:lang           -> layout publico por idioma
/:lang/          -> LandingPage
/:lang/how-it-works -> HowItWorksPage
/:lang/glossary   -> GlossaryIndexPage
/:lang/glossary/:slug -> GlossaryArticlePage
/:lang/about      -> AboutPage
/:lang/contact    -> ContactPage
/:lang/parceiros  -> PartnersPage
/:lang/partners   -> PartnersPage
/:lang/socios     -> PartnersPage
/:lang/parceiros/painel -> PartnerDashboardPage
/:lang/partners/dashboard -> PartnerDashboardPage
/:lang/socios/panel -> PartnerDashboardPage
/:lang/beta/:slug -> CampaignLandingPage
/:lang/campanha/:slug -> CampaignLandingPage
/:lang/bolao/copa -> WorldCupPoolCreatePage
/:lang/bolao/copa/entrar/:inviteToken -> WorldCupPoolInvitePage
/:lang/bolao/copa/meus-boloes -> WorldCupPoolMyPoolsPage
/:lang/bolao/copa/painel/:inviteToken -> WorldCupPoolParticipantDashboardPage
```

/:lang/bolao/copa/admin/:slug -> WorldCupPoolOrganizerPage
/app/* -> ProductApp, quando VITE_ENABLE_PRODUCT_APP=true
/app/account -> ProductAccountPage
/app/manual-analysis -> ProductManualAnalysisPage
/admin/* -> AdminApp, quando VITE_ENABLE_ADMIN_APP=true e /auth/me indica admin_access

O ProductApp e protegido por feature flags e, quando configurado, exige login. O AdminApp tambem depende de feature flag e valida /auth/me; ele renderiza a SPA administrativa apenas quando a resposta indica admin_access.

6 Fluxos funcionais principais

6.1 Fluxo publico de aquisicao

Visitante -> landing/glossario/conteudo

- > CTA para cadastro, campanha beta, campanha paga ou Bolao da Copa
- > eventos de telemetria/GA4 para origem, campanha, dispositivo e funil
- > signup/login via /auth/*
- > produto logado ou resgate de campanha

6.2 Fluxo de uso do produto

Usuario logado -> /app

- > GET /auth/me
- > GET /access/usage
- > GET /product/index
- > escolhe jogo
- > POST /access/reveal
 - se ja revelado: retorna sem cobrar novo credito
 - se limite diario disponivel: consome credito
 - se limite diario excedido e bonus existe: consome bonus
 - senao: retorna NO_CREDITS
- > frontend exhibe cards, odds, narrativa e decisao de oportunidade

6.3 Fluxo de assinatura

Usuario -> pagina de conta/upgrade

- > GET /billing/catalog
- > POST /billing/checkout/session
 - valida plano/ciclo/moeda/runtime
 - bloqueia checkout duplicado se assinatura ativa/trialing/past_due
 - cria ou reutiliza customer Stripe
 - cria Checkout Session em modo subscription e ui_mode=elements
- > Stripe confirma pagamento
- > POST /billing/webhooks/stripe[/sandbox|/live]

sincroniza assinatura
atualiza entitlement do usuario
grava eventos de billing/webhook

6.4 Fluxo de campanha beta/trial

Admin cria campanha -> /admin/access-campaigns

campos: slug, plano liberado, dias de trial, limites, janela de validade, oferta pos-trial

Visitante acessa /:lang/beta/:slug ou /:lang/campanha/:slug

-> GET /access/campaigns/{slug}

-> login/cadastro se necessario

-> POST /access/campaigns/{slug}/redeem

valida status, datas, limite, usuario, plano atual, trials anteriores e aprovacao

cria grant temporario em access.user_plan_grants

registra redemption

registra atribuicao de parceiro, quando aplicavel

cria elegibilidade de desconto pos-trial, quando configurada

6.5 Fluxo operacional de dados/odds/modelo

Cloud Scheduler -> POST /internal/ops/jobs/run

-> run_job

-> dispatcher

-> odds_refresh: busca odds e eventos

-> odds_resolve_batch: resolve times/eventos para fixtures core

-> models_ensure_1x2_v1: garante previsoes 1x2

-> snapshots_materialize: cria product.matchup_snapshot_v1

-> audit_sync_from_product_snapshots: alimenta auditoria

Frontend /app

-> GET /product/index

-> le product.matchup_snapshot_v1

7 Modelos, odds e snapshots

A camada de modelo combina historico, contexto e odds para produzir uma leitura 1x2 e, em areas especificas, mercados derivados de gols.

Componentes principais:

- score_engine_v1.py: motor Poisson independente para placares, com matriz de gols e probabilidades 1x2, BTTS, totals, team totals e placar exato.
- matchup_model_hist5_v1.py: modelo model_v1_hist5_decay, que mistura prior de liga, forma recente/historico e qualidade de cobertura.

- `model_registry.py`: seleciona versao ativa por `PREVIA_MODEL_VERSION`, com fallback para `model_v0`.
- `opportunity_decision_v1.py`: aplica regras de margem, confianca, numero de casas, frescor e outliers para rotular oportunidade ou cautela.
- `matchup_snapshot_builder_v1.py`: junta fixture, odds, previsao, decisao e narrativa em `product.matchup_snapshot_v1`. Qualidade/cobertura no modelo hist5: STRONG, OK, THIN, CUP_LIKE, INSUFFICIENT e UNKNOWN. Coberturas fracas reduzem blend/confianca e podem impedir recomendacao mesmo quando ha leitura estatistica.

A decisao de oportunidade nao e apenas “maior probabilidade”. Ela exige que a odd pague o risco com margem minima, que a fonte esteja fresca, que exista confianca suficiente e que os filtros de outlier/cobertura nao bloqueiem a recomendacao.

8 Planos, credits e entitlements

O codigo possui entitlements no backend e no frontend. O backend observado em `auth/service.py` usa os seguintes limites de referencia:

FREE -> `daily_limit 5, books_count 1, max_future_days 0`, sem chat/metrica/H2H

BASIC -> `daily_limit 10, books_count 1, max_future_days 3`

LIGHT -> `daily_limit 50, books_count 3, max_future_days 14`, exhibe `value/fair/edge/summary`

PRO -> `daily_limit 200, books_count 999, max_future_days 3650`, chat, metricas e H2H

No frontend, `entitlements.ts` tambem define `FREE_ANON` com 3 credits e alinha Free 5, Basic 10, Light 50 e Pro 200. Ha um arquivo `plan-config.ts` que parece conter configuracao legada/stale para Free anon/Free+ e deve ser tratado com cuidado antes de virar fonte de verdade.

Regras de revelacao importantes:

- Planos pagos (BASIC, LIGHT, PRO) possuem revelacoes persistentes enquanto o evento ainda e atual, evitando nova cobranca ao reabrir a analise.
- Usuarios free tem revelacoes mais ligadas ao dia/uso diario.
- Se o limite diario acabar, o sistema tenta usar credits bonus, quando existirem.
- A tabela de uso diario e travada com `FOR UPDATE` durante cobranca para evitar corrida. # Billing e Stripe

O modulo de billing aceita planos BASIC, LIGHT e PRO, ciclos `monthly`, `quarterly` e `annual`, moedas BRL e USD, e runtimes `sandbox` e `live`.

Checkout Stripe observado:

- Cria Checkout Session em `mode="subscription"`.
- Usa `ui_mode="elements"` e retorna `checkout_client_secret` ao frontend.

- Envia metadata com user_id, plan_code, billing_cycle, currency_code, plan_price_id, price_code, provider_price_id e billing_runtime.
- Usa allow_promotion_codes=True, permitindo código promocional manual.
- Bloqueia novo checkout se o usuário já tiver assinatura efetiva em estados como active, trialing ou past_due.
- Tem webhooks separados para rota genérica, sandbox e live. Ponto importante: o ZIP analisado mostra estrutura de elegibilidade de desconto pós-campanha, incluindo coupon/promotion code no banco, mas o checkout ainda não injeta automaticamente discounts ou promotion_code na sessão Stripe. Portanto, a oferta aparece como elegibilidade no sistema, mas não é aplicada automaticamente na assinatura sem patch adicional.

9 Campanhas, beta e oferta pos-trial

Campanhas são gerenciadas no Admin e consumidas publicamente por slug. Elas podem liberar trial temporário, controlar limite de resgates, janela de validade, planos/ciclos elegíveis e oferta pos-trial.

Campos e conceitos relevantes:

- Slug público e rotas `/:lang/beta/:slug` e `/:lang/campanha/:slug`.
- Plano liberado durante trial/beta.
- Duração do trial em dias.
- Limite total de resgates e contagem de redemptions.
- Início e expiração da campanha.
- Permissões para usuário existente, pagante ou usuário que já teve trial.
- Oferta pos-trial com percentual, duração, validade e elegibilidade por plano/ciclo.
- Associação opcional a parceiro/campaign link para atribuição. A aplicação do grant temporário passa por validações de status, datas, limite, usuário autenticado, plano atual, regras de trial anterior e aprovação. Quando elegível, insere grant temporário e registra redemption.

10 Parcerias e afiliados

O módulo de partnership cobre candidatura pública, aprovação/conversão no Admin, criação de parceiro, contrato, links de campanha e atribuições.

Rotas principais:

- POST `/public/partner-applications`: candidatura pública.
- GET/PATCH/POST `/admin/partners/applications*`: triagem e conversão no Admin.
- GET `/partner/me`: dados do parceiro autenticado.
- GET `/partner/campaigns`: campanhas/links associados ao parceiro.

- GET /partner/attributions: atribuicoes geradas por links. Regra comercial documentada em conversas anteriores: contrato de 12 meses, com comissao de referencia de 50% da receita liquida nos tres primeiros meses pagos de cada assinante indicado. Assinantes novos gerados dentro dos 12 meses entram na regra; depois do terceiro mes pago daquele assinante, a comissao cessa.

11 Bolao da Copa 2026

O Bolao da Copa e um modulo publico/recreativo, independente do produto principal de analise, mas util como funil organico. Ele permite criar bolao, convidar participantes, registrar palpites, travar palpites antes do jogo, calcular pontuacao e exibir ranking.

Fluxo do participante:

Criador -> /:lang/bolao/copa

-> POST /public/worldcup-pool/pools

-> recebe slug/token de convite

Participante -> /:lang/bolao/copa/entrar/:inviteToken

-> GET /public/worldcup-pool/invites/{invite_token}

-> cria/entra com nome, email e PIN

-> GET /participant/matches

-> PUT /participant/predictions/{match_id}

-> GET /participant/ranking

Regras observadas:

- Palpites travam uma hora antes do kickoff quando lock_at_utc nao esta preenchido.
- Ranking usa pontos totais e desempates por exatos, vencedores e gols de time.
- Modo classico: exato 5, resultado 3, bonus de placar de time 1, maximo 5.
- Modo progressivo por fase: aumenta pesos em mata-mata e final.
- Jobs sincronizam fixtures, resultados e emails/abertura de palpites. Observacao tecnica: para mata-mata, ha uma pendencia conceitual de produto sobre como tratar empate no tempo normal, prorrogacao, penaltis e classificado. Isso deve ser fechado antes de depender do modulo para fases eliminatorias complexas.

12 Ops, jobs e pipeline

A operacao de dados e baseada em ops_job_definitions, ops_job_runs, ops_job_attempts e ops_job_events. O caminho recomendado e Cloud Scheduler chamar /internal/ops/jobs/run com X-Ops-Trigger-Token. O endpoint interno chama o dispatcher, que resolve o job e grava eventos operacionais.

Jobs encontrados no dispatcher:

audit_sync_from_product_snapshots
models_ensure_1x2_v1
odds_catalog_sync
odds_league_autoclassify
odds_league_gap_scan
odds_refresh
odds_resolve_batch
oddsapi_run_controlled_enrichment
pipeline_run_all
snapshots_materialize
update_pipeline_run
update_pipeline_run_shard
worldcup_pool_results_sync
worldcup_pool_fixture_mapping_sync
worldcup_pool_predictions_open_email

Pipeline principal:

pipeline_run_all
-> odds_refresh
-> odds_resolve_batch
-> models_ensure_1x2_v1
-> snapshots_materialize
-> audit_sync_from_product_snapshots, quando aplicavel

O ZIP contem documentacao operacional indicando execucao periodica do pipeline_run_all durante o dia. O design atual e adequado para um monolito operacional, mas no medio prazo faz sentido separar superficie publica, admin e jobs em servicos distintos para reduzir risco e facilitar escala.

13 Banco de dados

O banco e organizado em schemas de dominio. Catalogo de tabelas encontrado em migrations SQL do ZIP:

[access]
- user_bonus_credit_balances
- user_bonus_credit_events
- user_daily_usage
- user_entitlements_snapshot
- user_manual_analyses
- user_manual_analysis_image_actions
- user_manual_analysis_image_requests
- user_manual_analysis_image_rows

- user_manual_analysis_image_usage_daily
- user_revealed_events

[app]

- user_identities
- user_product_preferences
- users

[auth]

- password_reset_tokens
- sessions

[billing]

- plan_prices
- plans
- subscription_change_requests
- subscription_events
- subscriptions
- webhook_events

[core]

- fixtures
- leagues
- team_season_stats
- teams

[odds]

- audit_event_predictions
- audit_predictions
- audit_result
- model_predictions_1x2
- odds_events
- odds_league_map
- odds_league_map_suggestions
- odds_snapshots_1x2
- odds_snapshots_market
- odds_sport_catalog
- provider_request_usage
- team_name_aliases
- team_name_resolution_log
- team_name_resolution_queue

[ops]

- job_runs
- ops_feature_flags
- ops_job_attempts
- ops_job_definitions
- ops_job_events
- ops_job_runs
- ops_job_scope_overrides

[partnership]

- partner_applications
- partner_attributions
- partner_audit_events
- partner_campaign_links
- partner_contracts
- partners

[product]

- matchup_snapshot_v1

[public]

- api_field_catalog
- api_raw

[public_site]

- beta_leads

[raw]

- api_responses
- backfill_checkpoint
- etl_runs

[telemetry]

- anonymous_identities
- events

[worldcup_pool]

- events
- matches
- participants
- pin_attempts
- pools
- predictions
- sessions

Algumas tabelas são referenciadas pelo código mas não apareceram como CREATE TABLE nos arquivos SQL analisados do ZIP. Isso não prova ausência em produção, mas indica que uma instalação limpa precisa conferir migrations ou dumps complementares. Exemplos: access.campaigns, access.campaign_redemptions, access.user_plan_grants, billing.user_discount_eligibilities, app.user_roles, app.role_capabilities, app.user_capability_overrides, app.internal_domain_rules, auth.security_flow_events e access.user_daily_credit_grants.

14 Providers externos e integrações

14.1 API-Football

Usada para fixtures, times, ligas, temporadas e resultados. O cliente envia x-apisports-key e retorna erro controlado quando a chave não está configurada.

14.2 The Odds API

Usada para catálogo de esportes, eventos e odds. O código tem funções para listar sports, buscar odds h2h e odds por evento. Há um ponto de atenção: foi observada duplicidade de método get_event_odds no cliente, com possível inconsistência de path dependendo do base_url.

14.3 OddsPapi

Integração de enriquecimento controlado. O cliente respeita caps de uso mensal e reserva antes de requisitar. A decisão de quais eventos enriquecer fica fora do client, o que preserva separação de responsabilidades.

14.4 Stripe

Usado para assinatura, checkout e webhooks. O sistema diferencia sandbox/live e armazena metadata suficiente para reconciliação.

14.5 OpenAI Vision

Usado pelo Image Import para extrair jogos/odds de prints. O módulo valida MIME, tamanho e dimensões com Pillow antes de chamar o modelo.

14.6 SMTP

Usado em fluxos de senha, notificações internas e possivelmente candidatura/parcerias. Variáveis SMTP aparecem separadas por contexto.

15 Image Import e análise manual

O Image Import permite que usuários elegíveis enviem prints de casas de apostas para extrair linhas de jogos e odds. Pelas flags e código, os planos permitidos são LIGHT e PRO.

Regras observadas:

- MIMEs suportados: JPEG, PNG e WebP.
- Tamanho maximo default: 8 MB.
- Imagem precisa ter lado longo minimo de 600 px.
- Light: default de 5 linhas por upload, 10 uploads/dia e cooldown de 20s.
- Pro: default de 15 linhas por upload, 30 uploads/dia e cooldown de 8s.
- Status de linha: READY, NEEDS_CONFIRMATION, UNSUPPORTED_MARKET, LOW_CONFIDENCE e UNREADABLE.
- Resolucao automatica depende de confianca alta; com country hint, limite em torno de 0.90; sem country hint, 0.92; acima de 0.75 pode pedir confirmacao. O fluxo tipico e: preview do print, confirmacao de linhas duvidosas, avaliacao em lote e consumo de creditos conforme regra do produto.

16 Telemetria e analytics

A API /telemetry/events coleta eventos anonimos ou associados a sessoes, com superficies como landing, auth, app, account, admin, public_embed e unknown. O Admin possui resumo em /admin/telemetry/anonymous-summary.

A estrategia documentada em conversas anteriores separa GA4 para macro-funil de aquisicao e telemetria interna para comportamento do produto. Eventos importantes incluem visualizacao de landing, fluxo de auth, checkout, revelacao anonima, revelacao logada, uso de creditos e jornadas pos-cadastro.

17 Deploy e configuracao

O deploy backend observado usa Cloud Build para construir uma imagem Docker e publicar no Cloud Run. O Dockerfile parte de Python 3.11 slim, instala dependencias e roda uvicorn app:app. O build referencia Cloud SQL e secrets para variaveis criticas.

O frontend usa Firebase Hosting. firebase.json define target de hosting, SPA rewrite e rewrite de API para Cloud Run. A separacao atual e suficiente para MVP/produto inicial, mas a tendencia de crescimento sugere separar workloads publicos, admin e jobs.

18 Variaveis de ambiente

Catalogo de nomes de variaveis encontrados no codigo. Valores nao sao reproduzidos.

18.1 Auth/Admin/Product access

ADMIN_AUTH_ENABLED

ADMIN_DEV_BYPASS_ENABLED

INTERNAL_STAFF_ADMIN_EMAILS

PRODUCT_AUTH_ENABLED

PRODUCT_DEV_AUTO_LOGIN_ENABLED
PRODUCT_PASSWORD_RESET_DEBUG_TOKEN_ENABLED
PRODUCT_PASSWORD_RESET_TTL_MINUTES
PRODUCT_SESSION_COOKIE_SECURE
PRODUCT_SESSION_TTL_DAYS

18.2 Core/Outros

DEV
FRONTEND_ALLOWED_ORIGINS
INTERNAL_NOTIFY_TO_EMAIL
PREVIA_ALLOW_PRODUCT_INDEX_DEBUG_REVEAL
PREVIA_MODEL_VERSION
PREVIA_SNAPSHOT_CALC_VERSION
PRODUCT_MANUAL_ANALYSIS_ENABLED

18.3 Emails/Parcerias

INTERNAL_SMTP_FROM_EMAIL
INTERNAL_SMTP_FROM_NAME
INTERNAL_SMTP_HOST
INTERNAL_SMTP_PASSWORD
INTERNAL_SMTP_PORT
INTERNAL_SMTP_TIMEOUT_SEC
INTERNAL_SMTP_USERNAME
INTERNAL_SMTP_USE_SSL
INTERNAL_SMTP_USE_TLS
PARTNER_APPLICATION_EMAIL_24H_LIMIT
PARTNER_APPLICATION_HASH_SALT
PARTNER_APPLICATION_IP_1H_LIMIT
PARTNER_APPLICATION_IP_24H_LIMIT
PARTNER_APPLICATION_NOTIFY_EMAIL
PRODUCT_SMTP_FROM_EMAIL
PRODUCT_SMTP_FROM_NAME
PRODUCT_SMTP_HOST
PRODUCT_SMTP_PASSWORD
PRODUCT_SMTP_PORT
PRODUCT_SMTP_TIMEOUT_SEC
PRODUCT_SMTP_USERNAME
PRODUCT_SMTP_USE_SSL
PRODUCT_SMTP_USE_TLS

18.4 Frontend Vite

VITE_API_BASE_URL
VITE_ENABLE_ADMIN_APP

VITE_ENABLE_PRODUCT_APP
VITE_ENABLE_PRODUCT_MANUAL_ANALYSIS_PAGE
VITE_ENABLE_PUBLIC_FREE_ANON_EMBED
VITE_ENABLE_PUBLIC_PRODUCT_LAYER
VITE_ENABLE_WORLDCUP_POOL
VITE_GA_MEASUREMENT_ID
VITE_PRODUCT_APP_REQUIRE_LOGIN
VITE_PRODUCT_AUTH_ENABLED
VITE_PRODUCT_DEV_AUTO_LOGIN_EMAIL
VITE_PRODUCT_DEV_AUTO_LOGIN_ENABLED
VITE_PRODUCT_DEV_AUTO_LOGIN_PLAN
VITE_PRODUCT_GOOGLE_AUTH_ENABLED
VITE_PRODUCT_GOOGLE_CLIENT_ID
VITE_PUBLIC_SITE_ORIGIN

18.5 Image import/OCR

IMAGE_IMPORT_ENABLED
IMAGE_IMPORT_LIGHT_COOLDOWN_SECONDS
IMAGE_IMPORT_LIGHT_MAX_ROWS
IMAGE_IMPORT_LIGHT_UPLOADS_PER_DAY
IMAGE_IMPORT_MAX_BYTES
IMAGE_IMPORT_PRO_COOLDOWN_SECONDS
IMAGE_IMPORT_PRO_MAX_ROWS
IMAGE_IMPORT_PRO_UPLOADS_PER_DAY
OPENAI_IMAGE_IMPORT_MAX_OUTPUT_TOKENS
OPENAI_IMAGE_IMPORT_TIMEOUT_SECONDS

18.6 Ops/Jobs

OPS_MANUAL_TRIGGER_ENABLED

18.7 Providers de dados e odds

ODDSPAPI_ENRICHMENT_ENABLED
ODDSPAPI_MONTHLY_HARD_CAP
ODDSPAPI_MONTHLY_RESERVE
ODDSPAPI_PRIMARY_BOOKMAKERS
ODDSPAPI_SECONDARY_BOOKMAKERS

18.8 World Cup Pool

WORLDCUP_POOL_ENABLED
WORLDCUP_POOL_JOIN_ENABLED
WORLDCUP_POOL_PREDICTIONS_ENABLED
WORLDCUP_POOL_PUBLIC_CREATE_ENABLED

WORLD_CUP_POOL_READONLY_ENABLED
WORLD_CUP_POOL_SESSION_TTL_DAYS

19 Segurança, privacidade e riscos operacionais

Guardrails positivos já presentes:

- Backend bloqueia produção com bypass admin ativo.
- Backend bloqueia produção com auto-login de produto ativo.
- Backend bloqueia produção com Admin Auth desabilitado.
- Trigger manual de ops em produção exige token configurado.
- Rotas admin dependem de capacidade/role e validação via /auth/me no frontend.
- Jobs internos usam token X-Ops-Trigger-Token. Riscos encontrados no pacote analisado:
- O ZIP contém artefatos que não deveriam circular em pacotes técnicos ou contexto de IA sem sanitização: .env, .env.local, .env.production, cloudrun.env.yaml, cookies/tokens temporários, .firebase, .git, possíveis artefatos de build e proxy binário.
- Segredos devem permanecer em Secret Manager ou ambiente seguro, nunca no repositório nem em ZIPs compartilháveis.
- Rotas de admin, ops e billing devem ter testes de permissão regressivos.
- Uma futura separação entre serviço público, serviço admin e serviço jobs reduz blast radius. # Pontos de atenção encontrados

Esta seção lista achados que merecem revisão antes de considerar o sistema completamente documentado e reproduzível.

- Checkout Stripe ainda não aplica automaticamente a elegibilidade de desconto de campanhas; atualmente o código permite códigos promocionais manuais.
- Frontend referencia PATCH /auth/profile, mas a varredura de rotas backend não encontrou esse endpoint. Pode ser rota removida, bug ou legado.
- Algumas tabelas referenciadas em código não apareceram como CREATE TABLE nas migrations do ZIP; validar migrations reais antes de instalar do zero.
- Cliente The Odds API contém duplicidade de método get_event_odds, com risco de sobrescrita silenciosa e path inconsistente.
- Em access_router.py, há duplicação aparente de um SELECT ... FOR UPDATE no fluxo de revelação, provavelmente sem efeito funcional grave, mas desnecessário.

- `frontend/src/product/config/plan-config.ts` parece legada ou desalinhada em relação a `entitlements.ts` e ao backend.
- O módulo Bolao ainda precisa fechar regra de mata-mata: tempo normal, prorrogação, penaltis e classificado.
- Pacotes técnicos devem ser gerados com allowlist e sem segredos, `node_modules`, `dist`, `.firebase` ou `.git`. # Recomendações de evolução
- Criar um documento ARCHITECTURE.md vivo no repositório, derivado deste PDF, com atualização obrigatória em patches estruturais.
- Adicionar um script de inventário automático para rotas, tabelas, env vars e jobs, gerando anexos atualizados.
- Centralizar a fonte de verdade de planos/creditos e remover configurações legadas.
- Implementar aplicação automática do desconto pos-trial no Stripe Checkout, com testes de campanha e fallback seguro.
- Criar testes de contrato para endpoints chamados pelo frontend, detectando rotas inexistentes como `/auth/profile`.
- Separar deploys ou ao menos endpoints sensíveis por `ingress/servico`: publico, admin e jobs.
- Criar rotina de sanitize para ZIPs enviados a pessoas/IAs.
- Formalizar ADRs para decisões importantes: modelo hist5, credits, billing, campanhas, Bolao e image import. # Guia rápido para uma IA analisar o sistema

Ordem recomendada de leitura para outra IA ou novo desenvolvedor:

- Comece por `backend/app.py` para entender routers e guardrails.
- Leia `frontend/src/routes.tsx` para mapear superfícies públicas, app e admin.
- Leia `backend/src/auth/service.py` e `frontend/src/product/config/entitlements.ts` para planos e permissões.
- Leia `backend/src/billing/service.py` e `backend/src/routes/billing_router.py` para assinatura Stripe.
- Leia `backend/src/access/campaigns.py` e `backend/src/api/admin_access_campaigns_router.py` para campanhas.
- Leia `backend/src/models/matchup_model_hist5_v1.py`, `score_engine_v1.py`, `opportunity_decision_v1.py` e `matchup_snapshot_builder_v1.py` para modelo/snapshots.
- Leia `backend/src/ops/job_dispatcher.py` e `backend/docs/OPS_JOBS.md` para jobs.
- Leia `backend/src/routes/worldcup_pool_router.py` e migrations `worldcup_pool` para o Bolao.

- Antes de rodar localmente, confira migrations e remova segredos/artefatos locais do pacote. # Apendice A - Catalogo de rotas backend

Catalogo gerado por varredura estatica de decorators FastAPI. Pode haver rotas condicionais por feature flag ou caminhos criados dinamicamente que exijam verificacao manual.

[backend/src/api/routes_admin.py]

```
GET /api/admin/catalog/fields
  handler: list_fields
GET /api/admin/contracts/export/apifootball
  handler: export_apifootball_contract
GET /api/admin/ingest/apifootball
  handler: ingest_apifootball_get
POST /api/admin/ingest/apifootball
  handler: ingest_apifootball
GET /api/admin/ingest/apifootball/callplan
  handler: ingest_apifootball_callplan_get
POST /api/admin/ingest/apifootball/callplan
  handler: ingest_apifootball_callplan
GET /api/admin/ingest/apifootball/fixtures-callplan
  handler: ingest_apifootball_fixtures_callplan_get
POST /api/admin/ingest/apifootball/fixtures-callplan
  handler: ingest_apifootball_fixtures_callplan
GET /api/admin/plan/apifootball/fixtures-callplan
  handler: preview_fixtures_callplan
GET /api/admin/raw/latest
  handler: latest_raw
GET /api/admin/raw/latest-by-instance
  handler: raw_latest_by_instance
GET /api/admin/status
  handler: admin_status
```

[backend/src/http/access_router.py]

```
GET /access/campaigns/{slug}
  handler: access_campaign_public
POST /access/campaigns/{slug}/redeem
  handler: access_campaign_redeem
POST /access/dev/reset-testing
  handler: access_dev_reset_testing
POST /access/reveal
  handler: access_reveal
GET /access/usage
  handler: access_usage
```

[backend/src/http/admin_access_campaigns_router.py]

GET /admin/access-campaigns
 handler: admin_access_campaigns_list
POST /admin/access-campaigns
 handler: admin_access_campaigns_create
GET /admin/access-campaigns/{campaign_id}
 handler: admin_access_campaigns_get
PUT /admin/access-campaigns/{campaign_id}
 handler: admin_access_campaigns_update
PATCH /admin/access-campaigns/{campaign_id}/status
 handler: admin_access_campaigns_status

[backend/src/http/admin_catalog_router.py]

POST /admin/odds/catalog/sync
 handler: admin_sync_odds_catalog

[backend/src/http/admin_odds_router.py]

POST /admin/odds/audit/backfill/fixtures
 handler: admin_odds_audit_backfill_fixtures
GET /admin/odds/audit/metrics/summary
 handler: admin_odds_audit_metrics_summary
POST /admin/odds/audit/refresh/results
 handler: admin_odds_audit_refresh_results
POST /admin/odds/audit/refresh_results
 handler: admin_odds_audit_refresh_results
POST /admin/odds/audit/snapshot
 handler: admin_odds_audit_snapshot
GET /admin/odds/league_map
 handler: admin_odds_league_map_list
GET /admin/odds/league_map/pending
 handler: admin_odds_league_map_pending
POST /admin/odds/league_map/{sport_key}/official_country
 handler: admin_odds_set_official_country
GET /admin/odds/markets/btts
 handler: admin_odds_market_btts
GET /admin/odds/markets/totals
 handler: admin_odds_market_totals
POST /admin/odds/matchup_snapshots/rebuild
 handler: admin_rebuild_matchup_snapshots
GET /admin/odds/queue
 handler: admin_odds_queue
GET /admin/odds/queue/intel
 handler: admin_odds_queue_intel

POST /admin/odds/refresh
 handler: admin_odds_refresh
POST /admin/odds/refresh_and_resolve
 handler: admin_odds_refresh_and_resolve
POST /admin/odds/resolve/batch
 handler: admin_odds_resolve_batch
GET /admin/odds/sports
 handler: admin_odds_list_sports
POST /admin/odds/team_resolution/approve
 handler: admin_team_resolution_approve
POST /admin/odds/team_resolution/dismiss
 handler: admin_team_resolution_dismiss
GET /admin/odds/team_resolution/pending
 handler: admin_team_resolution_pending
GET /admin/odds/team_resolution/search_teams
 handler: admin_team_resolution_search_teams
GET /admin/odds/upcoming
 handler: admin_odds_upcoming
GET /admin/odds/upcoming/intel_live
 handler: admin_odds_upcoming_intel_live
GET /admin/odds/upcoming/orchestrate
 handler: admin_odds_upcoming_orchestrate

[backend/src/http/admin_ops_router.py]

GET /admin/ops/leagues
 handler: admin_ops_list_leagues
POST /admin/ops/leagues/auto_resolve
 handler: admin_ops_auto_resolve_leagues
POST /admin/ops/leagues/toggle
 handler: admin_ops_toggle_league
POST /admin/ops/odds/enrichment/oddspapi/batch/1x2
 handler: admin_ops_oddspapi_batch_write_1x2_mapped_events
POST /admin/ops/odds/enrichment/oddspapi/diagnostics/auto-match
 handler: admin_ops_oddspapi_auto_match_diagnostic
POST /admin/ops/odds/enrichment/oddspapi/diagnostics/fixture-match
 handler: admin_ops_oddspapi_fixture_match_diagnostic
POST /admin/ops/odds/enrichment/oddspapi/diagnostics/odds
 handler: admin_ops_oddspapi_odds_diagnostic
GET /admin/ops/odds/enrichment/oddspapi/dry-run
 handler: admin_ops_oddspapi_enrichment_dry_run
GET /admin/ops/odds/enrichment/oddspapi/events/status
 handler: admin_ops_oddspapi_enrichment_events_status
POST /admin/ops/odds/enrichment/oddspapi/mappings/auto-confirm

handler: admin_ops_oddspapi_auto_confirm_mappings
POST /admin/ops/odds/enrichment/oddspapi/mappings/manual-confirm
handler: admin_ops_oddspapi_manual_confirm_mapping
POST /admin/ops/odds/enrichment/oddspapi/run
handler: admin_ops_oddspapi_run_controlled_enrichment
GET /admin/ops/odds/enrichment/oddspapi/status
handler: admin_ops_oddspapi_enrichment_status
POST /admin/ops/odds/enrichment/oddspapi/write/1x2
handler: admin_ops_oddspapi_write_1x2_snapshots
POST /admin/ops/odds/league_map/approve
handler: admin_ops_league_map_approve
POST /admin/ops/odds/league_map/autoclassify
handler: admin_ops_league_map_autoclassify
POST /admin/ops/odds/league_map/discover_candidates
handler: admin_ops_league_map_discover_candidates
POST /admin/ops/odds/league_map/gap_scan
handler: admin_ops_league_map_gap_scan
GET /admin/ops/odds/league_map/pending
handler: admin_ops_league_map_pending
GET /admin/ops/odds/league_map/suggestions
handler: admin_ops_league_map_suggestions
POST /admin/ops/odds/refresh
handler: admin_ops_odds_refresh
POST /admin/ops/odds/resolve
handler: admin_ops_odds_resolve
GET /admin/ops/pipeline/health
handler: admin_ops_pipeline_health
POST /admin/ops/pipeline/run
handler: admin_ops_pipeline_run
POST /admin/ops/pipeline/run_all
handler: admin_ops_pipeline_run_all
GET /admin/ops/pipeline/season-health
handler: admin_ops_pipeline_season_health
POST /admin/ops/pipeline/snapshots/cleanup-stale
handler: admin_ops_cleanup_stale_snapshots
GET /admin/ops/pipeline/snapshots/confidence-summary
handler: admin_ops_snapshots_confidence_summary
GET /admin/ops/pipeline/snapshots/low-confidence
handler: admin_ops_snapshots_low_confidence
GET /admin/ops/runs/recent
handler: admin_ops_runs_recent
GET /admin/ops/runs/{run_id}/events
handler: admin_ops_run_events

POST /admin/ops/snapshots/materialize
handler: admin_ops_snapshots_materialize

[backend/src/http/admin_partner_applications_router.py]

GET /admin/partners/applications
handler: admin_list_partner_applications
GET /admin/partners/applications/{application_id}
handler: admin_get_partner_application
PATCH /admin/partners/applications/{application_id}
handler: admin_update_partner_application
POST /admin/partners/applications/{application_id}/convert
handler: admin_convert_partner_application

[backend/src/http/admin_partners_router.py]

GET /admin/partners/{partner_id}
handler: admin_get_partner
GET /admin/partners/{partner_id}/campaign-links
handler: admin_list_partner_campaign_links
POST /admin/partners/{partner_id}/campaign-links
handler: admin_create_partner_campaign_link
POST /admin/partners/{partner_id}/campaign-links/{link_id}/activate
handler: admin_activate_partner_campaign_link
POST /admin/partners/{partner_id}/campaign-links/{link_id}/end
handler: admin_end_partner_campaign_link
POST /admin/partners/{partner_id}/campaign-links/{link_id}/pause
handler: admin_pause_partner_campaign_link

[backend/src/http/admin_router.py]

POST /admin/fixtures/refresh
handler: admin_refresh_fixtures
GET /admin/fixtures/upcoming
handler: admin_upcoming_fixtures
GET /admin/matchup/by-fixture
handler: admin_matchup_by_fixture
GET /admin/matchup/whatif
handler: admin_matchup_whatif
GET /admin/metrics/artifacts
handler: admin_metrics_artifacts
GET /admin/metrics/artifacts/raw
handler: admin_metrics_artifacts
GET /admin/metrics/overview
handler: admin_metrics_overview
GET /admin/odds/audit/reliability

handler: admin_odds_audit_reliability
GET /admin/odds/audit/reliability/by-league
handler: admin_odds_audit_reliability_by_league
GET /admin/odds/audit/reliability/events
handler: admin_odds_audit_reliability_events
GET /admin/odds/audit/shadow-snapshots
handler: admin_odds_audit_shadow_snapshots
GET /admin/odds/audit/shadow-snapshots.csv
handler: admin_odds_audit_shadow_snapshots_csv
POST /admin/odds/audit/sync-results
handler: admin_odds_audit_sync_results
GET /admin/odds/resolve
handler: resolve_odds_teams
GET /admin/team/summary
handler: admin_team_summary
GET /admin/teams
handler: admin_search_teams
GET /admin/teams/by-league-season
handler: admin_teams_by_league_season
GET /admin/teams/list
handler: admin_list_teams

[backend/src/http/admin_users_router.py]

GET /admin/users
handler: admin_list_users
POST /admin/users
handler: admin_create_user
GET /admin/users/{user_id}
handler: admin_get_user_detail
POST /admin/users/{user_id}/credits/grant
handler: admin_grant_user_credits
POST /admin/users/{user_id}/plan
handler: admin_set_user_plan
POST /admin/users/{user_id}/roles/upsert
handler: admin_upsert_user_role
POST /admin/users/{user_id}/status
handler: admin_set_user_status

[backend/src/http/auth_router.py]

GET /auth/account-preferences
handler: auth_account_preferences
PATCH /auth/account-preferences
handler: auth_account_preferences_patch

POST /auth/google/link
 handler: auth_google_link
POST /auth/google/login
 handler: auth_google_login
POST /auth/login
 handler: auth_login
POST /auth/logout
 handler: auth_logout
GET /auth/me
 handler: auth_me
POST /auth/password/change
 handler: auth_password_change
POST /auth/password/forgot
 handler: auth_password_forgot
POST /auth/password/reset
 handler: auth_password_reset
POST /auth/signup
 handler: auth_signup

[backend/src/http/billing_router.py]

GET /billing/catalog
 handler: billing_catalog
POST /billing/checkout/session
 handler: billing_checkout_session
GET /billing/checkout/session/status
 handler: billing_checkout_session_status
GET /billing/subscription
 handler: billing_subscription
POST /billing/subscription/cancel-renewal
 handler: billing_subscription_cancel_renewal
POST /billing/subscription/change-apply
 handler: billing_subscription_change_apply
POST /billing/subscription/change-cancel
 handler: billing_subscription_change_cancel
POST /billing/subscription/change-preview
 handler: billing_subscription_change_preview
POST /billing/subscription/change-schedule
 handler: billing_subscription_change_schedule
POST /billing/subscription/resume-renewal
 handler: billing_subscription_resume_renewal
POST /billing/webhooks/stripe
 handler: billing_stripe_webhook_legacy
POST /billing/webhooks/stripe/live

handler: billing_stripe_webhook_live
POST /billing/webhooks/stripe/sandbox
handler: billing_stripe_webhook_sandbox

[backend/src/http/internal_ops_router.py]

GET /internal/ops/jobs
handler: internal_ops_jobs
POST /internal/ops/jobs/run
handler: internal_ops_run_job

[backend/src/http/odds_router.py]

GET /odds/events
handler: list_odds_events
POST /odds/matchup/resolve
handler: resolve_matchup
GET /odds/matchup/snapshot
handler: get_matchup_snapshot
GET /odds/matchups
handler: list_matchups_cards
POST /odds/quote
handler: quote

[backend/src/http/partner_router.py]

GET /partner/attributions
handler: partner_console_attributions
GET /partner/campaigns
handler: partner_console_campaigns
GET /partner/me
handler: partner_console_me

[backend/src/http/product_index_router.py]

GET /product/index
handler: product_index

[backend/src/http/product_leagues_router.py]

GET /product/leagues
handler: product_leagues

[backend/src/http/product_manual_analysis_image_router.py]

POST /product/manual-analysis/image-import/evaluate-batch
handler: product_manual_analysis_image_evaluate_batch
POST /product/manual-analysis/image-import/preview
handler: product_manual_analysis_image_preview

POST /product/manual-analysis/image-import/rows/{row_id}/confirm
handler: product_manual_analysis_image_confirm_row

[backend/src/http/product_manual_analysis_router.py]

POST /product/manual-analysis/evaluate
handler: product_manual_analysis_evaluate
GET /product/manual-analysis/history
handler: product_manual_analysis_history

[backend/src/http/public_partner_applications_router.py]

POST /public/partner-applications
handler: create_partner_application

[backend/src/http/public_router.py]

POST /public/beta-leads
handler: create_beta_lead
POST /public/contact-messages
handler: create_contact_message

[backend/src/http/telemetry_router.py]

GET /admin/telemetry/anonymous-summary
handler: admin_anonymous_summary
POST /telemetry/events
handler: track_telemetry_event

[backend/src/http/worldcup_pool_router.py]

POST /public/worldcup-pool/access/login
handler: login_worldcup_pool_access
POST /public/worldcup-pool/access/pin-reset
handler: request_worldcup_pool_pin_reset
POST /public/worldcup-pool/access/reset-session
handler: reset_worldcup_pool_access_session
GET /public/worldcup-pool/invites/{invite_token}
handler: get_worldcup_pool_invite
POST /public/worldcup-pool/invites/{invite_token}/participant-login
handler: login_worldcup_pool_participant
POST /public/worldcup-pool/invites/{invite_token}/participant/logout
handler: logout_worldcup_pool_participant
GET /public/worldcup-pool/invites/{invite_token}/participant/matches
handler: list_worldcup_pool_participant_matches
GET /public/worldcup-pool/invites/{invite_token}/participant/me
handler: get_worldcup_pool_participant_dashboard
PUT /public/worldcup-pool/invites/{invite_token}/participant/predictions/{match_id}

```

    handler: upsert_worldcup_pool_prediction
GET /public/worldcup-pool/invites/{invite_token}/participant/ranking
    handler: get_worldcup_pool_participant_ranking
GET
/public/worldcup-pool/invites/{invite_token}/participant/ranking/{target_participant_id}/
locked-predictions
    handler: get_worldcup_pool_participant_locked_predictions
POST /public/worldcup-pool/invites/{invite_token}/participants
    handler: join_worldcup_pool
GET /public/worldcup-pool/me/pools
    handler: list_worldcup_pool_my_pools
POST /public/worldcup-pool/pools
    handler: create_worldcup_pool
GET /public/worldcup-pool/pools/{slug}/organizer
    handler: get_worldcup_pool_organizer_dashboard
POST /public/worldcup-pool/pools/{slug}/organizer-login
    handler: login_worldcup_pool_organizer
POST /public/worldcup-pool/pools/{slug}/organizer/logout
    handler: logout_worldcup_pool_organizer
POST /public/worldcup-pool/pools/{slug}/organizer/participant-session
    handler: create_worldcup_pool_organizer_participant_session
POST /public/worldcup-pool/pools/{slug}/organizer/participants/{participant_id}/remove
    handler: remove_worldcup_pool_participant
GET /public/worldcup-pool/pools/{slug}/organizer/session
    handler: get_worldcup_pool_organizer_session_status
GET /public/worldcup-pool/status
    handler: get_worldcup_pool_status

[backend/src/routes/debug_db.py]
GET /debug/db-status
    handler: db_status
GET /debug/pg-whoami
    handler: pg_whoami

```

20 Apendice B - Rotas frontend

```

/ -> redireciona para /pt
/:lang -> layout publico por idioma
/:lang/ -> LandingPage
/:lang/how-it-works -> HowItWorksPage
/:lang/glossary -> GlossaryIndexPage
/:lang/glossary/:slug -> GlossaryArticlePage
/:lang/about -> AboutPage

```

`/:lang/contact` -> `ContactPage`
`/:lang/parceiros` -> `PartnersPage`
`/:lang/partners` -> `PartnersPage`
`/:lang/socios` -> `PartnersPage`
`/:lang/parceiros/painel` -> `PartnerDashboardPage`
`/:lang/partners/dashboard` -> `PartnerDashboardPage`
`/:lang/socios/panel` -> `PartnerDashboardPage`
`/:lang/beta/:slug` -> `CampaignLandingPage`
`/:lang/campanha/:slug` -> `CampaignLandingPage`
`/:lang/bolao/copa` -> `WorldCupPoolCreatePage`
`/:lang/bolao/copa/entrar/:inviteToken` -> `WorldCupPoolInvitePage`
`/:lang/bolao/copa/meus-boloes` -> `WorldCupPoolMyPoolsPage`
`/:lang/bolao/copa/painel/:inviteToken` -> `WorldCupPoolParticipantDashboardPage`
`/:lang/bolao/copa/admin/:slug` -> `WorldCupPoolOrganizerPage`
`/app/*` -> `ProductApp`, quando `VITE_ENABLE_PRODUCT_APP=true`
`/app/account` -> `ProductAccountPage`
`/app/manual-analysis` -> `ProductManualAnalysisPage`
`/admin/*` -> `AdminApp`, quando `VITE_ENABLE_ADMIN_APP=true` e `/auth/me` indica `admin_access`

21 Apendice C - Tabelas por schema

[access]

- `user_bonus_credit_balances`
- `user_bonus_credit_events`
- `user_daily_usage`
- `user_entitlements_snapshot`
- `user_manual_analyses`
- `user_manual_analysis_image_actions`
- `user_manual_analysis_image_requests`
- `user_manual_analysis_image_rows`
- `user_manual_analysis_image_usage_daily`
- `user_revealed_events`

[app]

- `user_identities`
- `user_product_preferences`
- `users`

[auth]

- `password_reset_tokens`
- `sessions`

[billing]

- plan_prices
- plans
- subscription_change_requests
- subscription_events
- subscriptions
- webhook_events

[core]

- fixtures
- leagues
- team_season_stats
- teams

[odds]

- audit_event_predictions
- audit_predictions
- audit_result
- model_predictions_1x2
- odds_events
- odds_league_map
- odds_league_map_suggestions
- odds_snapshots_1x2
- odds_snapshots_market
- odds_sport_catalog
- provider_request_usage
- team_name_aliases
- team_name_resolution_log
- team_name_resolution_queue

[ops]

- job_runs
- ops_feature_flags
- ops_job_attempts
- ops_job_definitions
- ops_job_events
- ops_job_runs
- ops_job_scope_overrides

[partnership]

- partner_applications
- partner_attributions
- partner_audit_events

- partner_campaign_links
- partner_contracts
- partners

[product]

- matchup_snapshot_v1

[public]

- api_field_catalog
- api_raw

[public_site]

- beta_leads

[raw]

- api_responses
- backfill_checkpoint
- etl_runs

[telemetry]

- anonymous_identities
- events

[worldcup_pool]

- events
- matches
- participants
- pin_attempts
- pools
- predictions
- sessions

22 Apendice D - Inventario de codigo

.py	arquivos= 180	linhas= 68686
.json	arquivos= 170	linhas= 47001
.ts	arquivos= 88	linhas= 19659
.tsx	arquivos= 76	linhas= 35520
.sql	arquivos= 67	linhas= 4431
.css	arquivos= 10	linhas= 20058
.ps1	arquivos= 6	linhas= 660
.yaml	arquivos= 4	linhas= 354

.html arquivos= 2 linhas= 29
.md arquivos= 2 linhas= 852

23 Apendice E - Checklist de handoff tecnico

- Entregar ZIP sanitizado sem .env, cookies, tokens, .git, .firebase, node_modules, dist e artefatos temporarios.
- Incluir instrucoes de setup local com Python, Node, banco e variaveis necessarias sem valores sensiveis.
- Incluir ordem de migrations ou dump anonimo para instalacao limpa.
- Incluir tabela de feature flags e valores esperados por ambiente.
- Incluir matriz de permissoes por role/capability.
- Incluir rotas publicas, app, admin e internas separadas.
- Incluir jobs com schedule, payload default, token necessario e criterio de sucesso.
- Incluir plano de rollback para billing, campanhas e pipeline de snapshots.